Découverte du Framework LangChain

Initiation et introduction aux LLMs

Résumé de cours — Module IA Générative | Haythem GHAZOUANI

1. NLP — Traitement du Langage Naturel

Le **NLP (Natural Language Processing)** est une branche de l'Intelligence Artificielle dédiée à la compréhension, l'interprétation et la manipulation du langage humain par les machines.

- Auto-complétion (ex : Google Search)
- Traduction automatique (ex : Google Translate)
- Analyse de sentiments (positif / négatif / neutre)
- Résumé automatique de textes
- Réponses à des questions en langage naturel

Évolution du NLP — Frise chronologique :

Année	Modèle / Événement
1988	RNN (Recurrent Neural Networks)
1997	LSTM (Long Short-Term Memory)
2017	Transformers — article "Attention is all you need" (Google Brain)
2018	BERT (Google) — GPT dans "Improving Language Understanding by Generative Pre-Training"
2019	GPT-2
2020	GPT-3 (175 milliards de paramètres)
2021	GPT-3.5
2022	ChatGPT, InstructGPT, PaLM
2023+	LLaMA (Meta), Falcon, GPT-4, PaLM 2, BARD, Mistral Large

Focus — Architecture Transformer :

Introduite par Google en 2017, l'architecture Transformer permet un **traitement parallèle** du langage et une **compréhension contextuelle** améliorée grâce au mécanisme d'attention (Multi-Head Attention). Elle est à la base de tous les grands modèles de langage modernes.

2. Les LLMs — Grands Modèles de Langage

Un **LLM (Large Language Model)** est un modèle d'apprentissage en profondeur (deep learning) basé sur l'architecture des Transformers, capable de comprendre et de générer du texte de manière proche du langage humain.

Capacité	Description
Génération de texte	Produire des articles, histoires et contenus écrits de manière cohérente et créative
Traduction automatique	Traduire des textes entre différentes langues en conservant le sens et le contexte
Résumé automatique	Condenser de longs documents en résumés concis sans perdre les informations essentielles
Réponses aux questions	Fournir des réponses précises à des questions posées en langage naturel

Exemples de LLMs :

Modèle	Description
GPT-3 (OpenAI)	175 milliards de paramètres, célèbre pour sa capacité à générer du texte de haute qualité
BERT (Google, 2018)	L'un des premiers LLMs multi-tâches : compréhension de texte, question-réponse
LLaMA (Meta)	Se distingue par sa performance et son accessibilité (open-source)
Mistral Large	Modèle français d'IA générative développé par Mistral AI

3. LangChain — Définition et intérêt

LangChain est un framework open-source conçu pour développer des applications alimentées par des **modèles de langage**. Il facilite leur intégration dans diverses applications en orchestrant les interactions entre le LLM, les données et les services externes.

Pourquoi utiliser LangChain ?

1. Flexibilité	2. Automatisation	3. Services tiers	4. Communauté
Intégration simplifiée d'OpenAI, HuggingFace et d'autres LLMs dans n'importe quelle application.	Orchestration d'une séquence d'actions complexes autour d'un modèle de langage.	Accès aux API Google, LinkedIn, Twitter, SerpAPI, Wolfram, Notion, etc.	LangChain Hub : partage d'agents, de codes et de prompts déjà créés par la communauté.

4. Concepts clés de LangChain

4.1 — LLMChain

L'**LLMChain** est le composant central de LangChain. Il permet de combiner **plusieurs composants** ensemble (LLM, prompt, mémoire, outils...) pour créer une **application cohérente et complexe**. Le flux est : entrée utilisateur → Étape 1 → Étape 2 → ... → Étape N → sortie finale. Les modules disponibles incluent : Google Drive, Cohere LLMs, Embeddings, CharacterTextSplitter, Google Search, SerpAPI, PDF, Agents, Memory, Indexes, Question Answering.

4.2 — ChromaDB

ChromaDB est une base de données vectorielle utilisée pour stocker et rechercher des embeddings. Elle joue un rôle clé dans le pipeline RAG :

- Les requêtes de l'application sont converties en embeddings (vecteurs numériques)
- ChromaDB stocke les documents et leurs embeddings associés
- Lors d'une requête, ChromaDB retrouve les documents les plus similaires sémantiquement
- Le contexte pertinent est transmis à la fenêtre de contexte du LLM pour générer la réponse

4.3 — Agents

Un **Agent** est comme un robot autonome qui effectue des actions en votre nom en exploitant des **outils** pour répondre à une tâche. Il suit le cycle :

- Query → Agent reçoit la question
- LLM Call → L'agent consulte le LLM pour raisonner (Thought)
- Parsing → L'agent analyse le raisonnement pour choisir l'outil approprié
- Tool Execution → Exécution de l'outil (Google Search, SerpAPI, Wolfram...)
- Output → Si le résultat est satisfaisant (OK), l'agent renvoie la réponse ; sinon il reboucle

4.4 — Document Loader

Le **Document Loader** permet à LangChain de charger et d'ingérer des données depuis diverses sources (PDF, CSV, pages web, bases de données...) pour les rendre exploitables par le LLM dans une chaîne de traitement.

4.5 — RAG (Retrieval-Augmented Generation)

Le **RAG** est une technique qui enrichit le prompt envoyé au LLM avec un **contexte récupéré dynamiquement** depuis une base de données vectorielle. Cela permet au modèle de répondre avec des informations spécifiques et actualisées sans avoir été ré-entraîné.

- 1) Le texte est découpé en chunks (morceaux)
- 2) Chaque chunk est encodé par un modèle d'embedding → vecteur numérique
- 3) Les vecteurs sont stockés dans un vectorstore (ex : ChromaDB)

- 4) Lors d'une requête, la query est aussi encodée et comparée aux vecteurs stockés
- 5) Les chunks les plus proches sémantiquement sont récupérés comme contexte
- 6) Ce contexte + la query forment le prompt final envoyé au LLM

Notion d'embeddings : *un embedding est une représentation vectorielle d'un texte dans un espace multidimensionnel. Des textes sémantiquement proches auront des vecteurs proches dans cet espace, permettant une recherche par similarité efficace.*

5. Exemple de code — Pipeline RAG avec LangChain

Voici la structure de base d'une application LangChain utilisant un LLM Groq et un retriever vectoriel (RetrievalQA) :

```
# Initialisation du modele Groq
groq_llm = ChatGroq()

# Creation du prompt pour Groq
prompt_template = "Repondez a la question suivante en vous basant sur le contexte:
{context}"

# Definition de la chaine QA avec le retriever
qa_chain = RetrievalQA.from_chain_type(
llm=groq_llm,
chain_type="stuff",
retriever=vectorstore.as_retriever(),
return_source_documents=True
)
```

6. Application pratique — IceBreaker (boutique de mode)

Exemple concret d'application LangChain développée dans ce cours : un assistant intelligent pour une boutique de mode homme/femme 2025.

- Recevoir une requête (question) sur les articles disponibles dans la boutique
- Rechercher dans les données de la boutique (vectorstore)
- Générer une réponse sur les articles disponibles avec possibilité de conseils personnalisés

Composant	Rôle
Document Loader	Chargement du catalogue articles de la boutique (CSV, PDF...)
Text Splitter	Découpage du catalogue en chunks exploitables
Embeddings	Conversion des chunks en vecteurs numériques

ChromaDB	Stockage et indexation des vecteurs
Retriever	Recherche des articles les plus pertinents selon la requête utilisateur
LLM (Groq/ChatGroq)	Génération de la réponse enrichie avec le contexte récupéré

7. Ce qu'il faut retenir

Concept	Essentiel à retenir
NLP	Branche de l'IA pour comprendre et générer du langage humain.
Transformer	Architecture de base de tous les LLMs modernes (attention mechanism).
LLM	Modèle deep learning capable de comprendre et générer du texte (GPT, BERT, LLaMA...).
LangChain	Framework pour construire des applications LLM : chaînes, agents, mémoire, outils.
RAG	Technique enrichissant le LLM avec un contexte dynamique récupéré depuis un vectorstore.
ChromaDB	Base de données vectorielle pour stocker et rechercher des embeddings.
Agent	Entité autonome qui raisonne (LLM) et agit (outils) en boucle pour résoudre une tâche.